PDV WEB · DOCUMENTAÇÃO DE TESTES

Relatório de Execução de Testes

Campo	Valor
Identificador	REX-PDV-001 · versão 1.2 (Ciclo 1.1 — robustez, seção 11; Ciclo 1.2 — banco de dados, seção 12)
Plano de origem	PT-PDV-001
Item testado	`pdv-web-api` 2.3.0 (backend REST)
Execução oficial — Ciclo 1	26/10/2026, 15:31:32 – 15:33:35 · `./mvnw clean verify` · 1 min 59 s · 265/265
Execução oficial — Ciclo 1.1	28/10/2026, 11:09:32 – 11:12:26 · `./mvnw clean verify` · 2 min 50 s · 313/313
Execução oficial — Ciclo 1.2 (banco)	**28/10/2026, 11:49:26 – 11:53:05** · `./mvnw clean verify` · 3 min 34 s · **320/320** (as evidências atuais são desta execução)
Ambiente	Windows 10 Pro · OpenJDK 25.0.3 · Maven 3.9.16 · Spring Boot 4.1.2 · PostgreSQL 17 embarcado (Plano, seção 8)
Evidências	evidencias/ — lista de casos executados, cobertura, desempenho e relatório BDD

1. Resumo executivo

> **Atualização de 28/10/2026 — Ciclo 1.1 (robustez).** As seções 1 a 9 registram o Ciclo 1 como foi executado. O Ciclo 1.1 corrigiu os riscos residuais da seção 7 (4 defeitos: DEF-WEB-003 a 006), criou **37 casos novos** e terminou com **313/313 execuções aprovadas**, cobertura de **96,8% das linhas e 87,6% dos ramos** e força de mutação de **100% em 12 classes**. Detalhes na seção 11. O **Ciclo 1.2** endureceu o banco de dados (gatilhos de imutabilidade, papéis com menor privilégio, Docker com backup): **320/320** — seção 12.

O Ciclo 1 executou **todos os 211 casos automatizados planejados** (195 casos CT e 16 cenários BDD), que somam **265 execuções**. **Todas foram aprovadas** na regressão final. Ao longo do ciclo, os testes encontraram **2 defeitos no produto**, ambos de segurança (DEF-WEB-001 e DEF-WEB-002), e **1 defeito num script de teste** (DEF-TST-001). Os três foram corrigidos e retestados.

Destaques:

- **Integridade de estoque comprovada sob concorrência:** 20 vendas simultâneas do último lote de 5 unidades resultaram em exatamente 5 vendas e estoque 0.
- **Regras financeiras validadas com os exemplos oficiais:** venda de R$ 215,00 com comissão de R$ 10,75; comissão acumulada de R$ 225,00; saldo de caixa de R$ 300,00.
- **Cobertura de código:** 94,1% das linhas e 81,2% dos ramos.
- **Força dos testes unitários (teste de mutação):** 100% dos mutantes cobertos foram detectados.
- **Desempenho:** p95 máximo de 36 ms, contra o limite de 3.000 ms do VAL-07.

Todos os critérios de saída do ciclo foram atendidos (seção 6). **Recomendação:** o backend está apto para o Ciclo 2 (integração com o front-end), mas **não deve ir para produção** antes de concluir as pendências PEND-03, PEND-04, PEND-06 e PEND-07 (seção 8).

2. Escopo executado e desvios em relação ao plano

Item	Planejado	Executado	Desvio
Casos automatizados (CT + BDD)	211	211	—

Item	Planejado	Executado	Desvio
Execuções automatizadas	265	265	—
Itens manuais e exploratórios (roteiro)	16	0	Sem PostgreSQL com interface gráfica, sem Docker e sem sessão com testadores neste ambiente → PEND-01, 02, 03
Reteste do desktop	7 (RT-DSK)	0 (só verificação de compilação)	PEND-02
Teste de mutação	Regras de negócio	11 classes	Executado só com os testes unitários, por tempo (ver 5.3)

Casos novos criados durante o ciclo: **CT-SEG-009** (originado da análise de risco da sessão EXP-02) e **CT-VEN-U18, CT-VEN-U19, CT-DOC-012** (originados do teste de mutação).

3. Resultados por nível e suíte

3.1 Por nível de teste

Nível	Casos distintos	Execuções	Aprovadas	Reprovadas	Taxa de aprovação
Unitário (JUnit 6 + Mockito)	65	96	96	0	100%
Componente / contrato HTTP (WebMvcTest)	12	12	12	0	100%
Integração e sistema (Spring Boot + PostgreSQL)	118	139	139	0	100%
Aceitação BDD (Cucumber / Gherkin)	16	18	18	0	100%
Total	**211**	**265**	**265**	**0**	**100%**

3.2 Por suíte

Suíte	Execuções	Resultado
Validação de CPF/CNPJ (Documentos)	21	21 ✅
Cálculo financeiro da venda (CalculadoraVenda)	15	15 ✅
Regras do produto e estoque	14	14 ✅
Paginação e convenções do contrato	11	11 ✅
Regras do cupom de desconto	10	10 ✅
Serviço de vendas (Mockito)	10	10 ✅
Transições de estado de Caixa e Venda	7	7 ✅
Serviço de autenticação (Mockito)	4	4 ✅
Serviço de caixa (Mockito)	4	4 ✅
Contrato HTTP de /produtos (WebMvcTest)	12	12 ✅
Produtos e estoque (integração)	23	23 ✅
Registro de vendas + Exemplos das Regras de Sistema (integração)	20	20 ✅
Segurança e controle de acesso (integração)	17	17 ✅
Controle de caixa (integração)	12	12 ✅
Cancelamento, histórico e comissões (integração)	12	12 ✅
Clientes (integração)	11	11 ✅
Autenticação e sessão (integração)	9	9 ✅

Suíte	Execuções	Resultado
Usuários e funcionários (integração)	9	9 ✅
Cupons de desconto (integração)	6	6 ✅
Integridade do banco de dados (integração)	5	5 ✅
Auditoria (integração)	4	4 ✅
Concorrência de vendas e cancelamentos (integração)	4	4 ✅
Desempenho — VAL-07 (integração)	4	4 ✅
Smoke test	3	3 ✅
Cenários BDD (Gherkin)	18	18 ✅ (130/130 passos)

O resultado individual de cada caso, com tempo de execução, está em evidencias/casos-executados.tsv.

4. Histórico das execuções do ciclo

Cada problema encontrado foi analisado, corrigido e seguido de reteste do bloco e da regressão completa (Tutorial, etapa 7).

#	Execução	Resultado	Ação
E1	Smoke test (subida da aplicação + banco embarcado)	3/3	Ambiente validado
E2	Unitários	93/93	—
E3	Contrato HTTP (WebMvcTest)	**12 erros** — contexto não inicializou	INC-01 (dependência de teste do Spring Boot 4) → 12/12
E4	Autenticação + segurança	25 executados, **5 falhas** (CT-SEG-005)	**DEF-WEB-001** corrigido → reteste 28/28
E5	Cadastros (produtos, clientes, cupons, usuários) + segurança	65/65	—
E6	Caixa, vendas, cancelamento, comissões, auditoria	48/48	—
E7	Concorrência, desempenho, banco	13/13	—
E8	BDD	18 executados, **3 falhas** (BDD-DES R5–R7)	**DEF-TST-001** (script de teste) corrigido → 18/18
E9	Teste de mutação — rodadas 1 a 3	Força 91,4% → 98,8% → **100%**	Lacunas MUT-01 a MUT-07 fechadas com casos novos e asserções reforçadas
E10	Regressão completa nº 1	264/264	—
E11	CT-SEG-009 (novo, da análise da sessão EXP-02)	**1 falha**	**DEF-WEB-002** corrigido → reteste 47/47 (segurança, autenticação, usuários, contrato HTTP)
E12	Regressão completa nº 2 (14:21)	265/265 · cobertura atendida · BUILD SUCCESS	Evidências sobrescritas pela E13
E13	**Regressão completa final (oficial, 15:31)**	**265/265** · cobertura atendida · BUILD SUCCESS	Base deste relatório e das evidências

5. Métricas (Tutorial, seção 14)

5.1 Execução e defeitos

Métrica	Cálculo	Valor
Taxa de aprovação (final)	265 aprovados ÷ 265 executados × 100	**100%**
Taxa de reprovação (final)	0 ÷ 265 × 100	0%
Casos automatizados executados	211 ÷ 211	100%
Casos planejados executados (inclui manuais)	211 ÷ 227	93,0%
Falhas na primeira execução dos casos	9 execuções em 5 casos/cenários: CT-SEG-005 (×5, DEF-WEB-001), BDD-DES R5–R7 (DEF-TST-001), CT-SEG-009 (DEF-WEB-002)	9 ÷ 265 = 3,4%
Defeitos encontrados no backend (produto)	—	2 (1 Alta, 1 Média)
Defeitos no script de teste	—	1 (Baixa)
Defeitos reabertos	—	0
Densidade de defeitos do backend	2 defeitos ÷ 1,426 KLOC executáveis	1,4 defeito/KLOC
Cobertura de requisitos (completa)	33 ÷ 39	84,6%
Cobertura de requisitos (≥ 1 caso aprovado)	38 ÷ 39	97,4% (100% do escopo do ciclo)

> Como alerta o Tutorial: 100% de aprovação significa que **os comportamentos cobertos** funcionam, não que o sistema não tem defeitos. Por isso este relatório traz também a força dos testes (5.3) e os riscos residuais (seção 7).

5.2 Cobertura de código (JaCoCo)

Contador	Cobertos / total	Percentual	Meta
Instruções	7.417 / 7.925	93,6%	—
Linhas	1.342 / 1.426	**94,1%**	≥ 80% ✅
Ramos (decisões)	346 / 426	**81,2%**	≥ 70% ✅
Métodos	473 / 514	92,0%	—

Pacote	Linhas	Ramos	Observação
`venda`	98,9%	90,5%	
`caixa`	98,1%	100,0%	
`auth`	98,3%	95,0%	
`comum`	100,0%	92,9%	
`produto`	94,8%	77,6%	
`seguranca`	94,8%	68,2%	⚠ abaixo da meta de ramos — limpeza da lista de revogação e caminhos de claim inválida
`auditoria`	94,6%	**50,0%**	⚠ filtros de data do `GET /auditoria` sem teste
`cupom`	93,3%	71,1%	

Pacote	Linhas	Ramos	Observação
`usuario`	87,7%	79,4%	`InicializadorGerente`: caminhos de configuração inválida
`cliente`	87,3%	72,1%	PATCH trocando tipo/documento
`comum.erro`	85,9%	70,0%	Tratadores de erros raros (ex.: parâmetro obrigatório ausente)
`br.com.empresa.pdv`	33,3%	—	Só o método `main` (não executado em teste)

Arquivo completo: evidencias/cobertura-jacoco.csv. HTML: `pdv-web-api/target/site/jacoco/index.html`.

5.3 Teste de mutação (PIT)

O PIT altera o código de produção ("mutantes": troca `<` por `<=`, remove chamadas, inverte condições) e confere se algum teste falha. Mutante que sobrevive indica teste que não detectaria aquele erro.

Escopo: 11 classes de regra de negócio (`CalculadoraVenda`, `Venda`, `VendaService`, `Caixa`, `CaixaService`, `Cupom`, `Produto`, `Documentos`, `Dinheiro`, `Paginacao`, `AuthService`), avaliadas pelos testes **unitários**.

Rodada	Mortos	Sobreviventes	Sem cobertura unitária	Força dos testes (mortos ÷ cobertos)
1	148	14	112	91,4%
2	163	2	109	98,8%
3 (final)	**165**	**0**	109	**100%** ✅ (meta ≥ 80%)

- As lacunas da rodada 1 estão em Defeitos, seção 5 (MUT-01 a MUT-07). A mais grave: remover a baixa de estoque da venda não reprovava nenhum teste unitário.
- Os 109 mutantes "sem cobertura unitária" ficam em fluxos testados só na integração (listagens, cancelamento, comissões, fechamento de caixa). O PIT não roda os testes de integração por custo de tempo. Escore sobre todos os mutantes gerados: 165 ÷ 274 = 60,2%.
- A correção do DEF-WEB-002 (feita depois das rodadas) não alterou nenhuma das 11 classes avaliadas.

5.4 Desempenho (VAL-07 — limite de 3.000 ms)

Caso	Operação	Amostras	Média	Mediana	p95	Máximo	Resultado
CT-PERF-001	`GET /produtos` (1.000 produtos, limite 100)	20	23,4 ms	23 ms	34 ms	42 ms	✅
CT-PERF-002	`GET /produtos?nome=…`	20	12,8 ms	13 ms	15 ms	15 ms	✅
CT-PERF-003	`POST /vendas` com 10 itens	30	26,8 ms	26 ms	36 ms	46 ms	✅
CT-PERF-004	`GET /produtos/{id}`	30	8,1 ms	8 ms	11 ms	11 ms	✅

Medição em ambiente local, sem rede entre cliente e servidor. A margem é grande (p95 98% abaixo do limite), mas não substitui o teste de carga na infraestrutura Docker/AWS (PEND-04). Fonte: evidencias/desempenho.csv.

5.5 Concorrência

Caso	Cenário	Resultado obtido
CT-CONC-001	20 vendas simultâneas de 1 unidade, estoque 5	5 × 201, 15 × 422, estoque final 0, 5 vendas gravadas ✅
CT-CONC-002	2 cancelamentos simultâneos da mesma venda	1 × 200, 1 × 422, estoque estornado uma única vez ✅
CT-CONC-003	5 sangrias simultâneas de R$ 30 com saldo de R$ 100	3 aceitas (R$ 90), 2 rejeitadas ✅
CT-CONC-004	4 aberturas de caixa simultâneas pelo mesmo operador	1 caixa criado ✅

6. Avaliação dos critérios de saída

#	Critério	Meta	Obtido	Situação
CS-1	Casos P1 executados e aprovados	100%	100%	✅ Atendido
CS-2	Taxa de aprovação geral	≥ 95%	100%	✅ Atendido
CS-3	Defeitos Críticos/Altos abertos (backend)	0	0	✅ Atendido
CS-4	Requisitos do escopo com ≥ 1 caso aprovado	100%	100% (38/38)	✅ Atendido
CS-5	Cobertura de código	≥ 80% linhas, ≥ 70% ramos	94,1% / 81,2%	✅ Atendido
CS-6	Tempo de resposta (p95)	< 3.000 ms	máx. 36 ms	✅ Atendido (ambiente local)
CS-7	Força dos testes unitários	≥ 80%	100%	✅ Atendido

7. Riscos residuais

Risco	Impacto	Recomendação	Situação após o Ciclo 1.1
Ramos pouco cobertos em `auditoria` (50%) e `seguranca` (68,2%)	Médio	Criar casos para os filtros de data da auditoria e para claims inválidas no token (Ciclo 2)	✅ Tratado: `auditoria` 87,5%, `seguranca` 73,9%
Revogação de tokens em memória	Médio (com várias instâncias)	Persistir a lista de revogação antes de escalar horizontalmente	✅ Corrigido (DEF-WEB-003)
Login sem limite de tentativas	Médio (força bruta)	Rate limiting / bloqueio temporário por CPF e IP	✅ Corrigido (DEF-WEB-004 e DEF-WEB-005)
Base de cálculo da comissão (líquido × bruto) não confirmada	Alto (financeiro)	Decisão da equipe (PEND-06); a mudança é localizada em `CalculadoraVenda`	✅ Configurável (`PDV_COMISSAO_BASE`); a equipe só escolhe o valor
Desktop legado sem reteste funcional	Baixo (o sistema será substituído)	Executar RT-DSK-01 a 07 se o desktop continuar em uso	Aberto (PEND-02)

8. Pendências

ID	Pendência	Responsável	Bloqueia produção?	Situação em 28/10/2026
PEND-01	Executar as sessões exploratórias EXP-01 a EXP-06	Equipe de testes	Não	Aberta. O modo demonstração (`demo.cmd` / `demo.sh` + Swagger UI) já permite executá-las sem Docker
PEND-02	Reteste manual das correções do desktop (RT-DSK-01 a 07)	Equipe	Não	Aberta
PEND-03	Subir com Docker Compose e executar RT-DEP-01 a 07	Equipe / infraestrutura	**Sim**	Parcial: RT-DEP-01 a 07 **aprovados no Docker** em 28/10/2026 (DEF-WEB-007/008/009 corrigidos); falta repetir no servidor AWS
PEND-04	Teste de carga (JMeter) na instância AWS EC2 — VAL-07 no ambiente alvo	Infraestrutura	**Sim**	Parcial: carga local com 20 operadores simultâneos aprovada (CT-PERF-005); falta o ambiente alvo
PEND-05	Executar a bateria num pipeline Linux (CI)	Equipe	Não	Pipeline criado (`.github/workflows/ci.yml` , Ubuntu + JDK 25); roda no primeiro push ao GitHub
PEND-06	Confirmar a base de cálculo da comissão (ADR D-04 / DOC-04)	Responsáveis pelo projeto	~~Sim~~ Não	✅ Resolvida por configuração (`PDV_COMISSAO_BASE` , padrão `LIQUIDO`)

ID	Pendência	Responsável	Bloqueia produção?	Situação em 28/10/2026
PEND-07	Configurar HTTPS/TLS no ambiente (VAL-06)	Infraestrutura	**Sim**	Parcial: proxy TLS (Caddy, perfil `tls`) pronto e aprovado localmente em RT-DEP-05; falta o domínio e o certificado Let's Encrypt na AWS
PEND-08	Implementar limite de tentativas de login	Equipe	Recomendado	✅ Concluída (DEF-WEB-004/005)

9. Conclusão

O backend do PDV Web cumpre, no ambiente de teste, as regras de negócio, as validações de estoque, caixa, desconto e cancelamento, e o controle de acesso previsto na documentação. Os casos reais das Regras de Sistema e os exemplos do contrato v2.3 passaram exatamente como especificados. Os três defeitos encontrados no ciclo foram fechados com reteste e regressão completa.

Decisão recomendada: aprovar o encerramento do Ciclo 1 e liberar o backend para integração com o front-end (Ciclo 2). A liberação para produção depende das pendências marcadas como bloqueantes na seção 8.

10. Aprovação

Papel	Nome	Data	Decisão
Responsável pelo projeto			
Revisor de testes			

11. Ciclo 1.1 — Robustez

Objetivo: eliminar os riscos residuais do Ciclo 1 (seção 7) e as lacunas de teste conhecidas, deixando o backend tolerante a reinícios, a ataques de força bruta e a picos de acesso. **Execução oficial:** 28/10/2026, 11:09:32 – 11:12:26 · `./mvnw clean verify` · BUILD SUCCESS.

11.1 O que mudou no produto

Ponto fraco do Ciclo 1	Correção	Defeito	Casos que comprovam
Revogação de tokens em memória (perdida no reinício, não compartilhada entre instâncias)	Tabela `tokens_revogados` (migração V3) + limpeza agendada	DEF-WEB-003	CT-SEG-011, CT-SEG-012
Login sem limite de tentativas	Bloqueio temporário por CPF (5 falhas) e por IP (20 falhas), 429 + `Retry-After`, auditoria `ACESSO_BLOQUEADO`	DEF-WEB-004	CT-SEG-013 a 016, CT-SEG-019, CT-AUTH-U05/U06
Senha do gerente no desconto sem limite	Bloqueio por vendedor (5 falhas), auditoria `AUTORIZACAO_FALHA`	DEF-WEB-005	CT-SEG-017, CT-VEN-U24
Lentidão na primeira rajada após a subida	Aquecimento do pool e das consultas antes da prontidão	DEF-WEB-006	CT-PERF-005
Sem HTTPS	`PDV_EXIGIR_HTTPS=true` redireciona HTTP → HTTPS; HSTS de 1 ano; cabeçalhos de segurança	—	CT-SEG-018, CT-SEG-020
Base da comissão indefinida	`PDV_COMISSAO_BASE` = `LIQUIDO` (padrão) ou `BRUTO`	DOC-04	CT-CALC-009, CT-COM-004
Token expirado sem teste	Validação de expiração com relógio da aplicação e tolerância de 30 s	—	CT-SEG-010
API nunca executada fora dos testes	Modo demonstração (`demo.cmd` / `demo.sh`), Swagger UI, desligamento gracioso	—	Smoke em servidor real (14 passos)

Ponto fraco do Ciclo 1	Correção	Defeito	Casos que comprovam
Sem pipeline Linux	`.github/workflows/ci.yml` (Ubuntu, JDK 25, relatórios publicados)	—	Executa no primeiro push

11.2 Resultados por nível

Nível	Casos distintos	Execuções	Aprovadas	Ciclo 1 → 1.1
Unitário (JUnit 6 + Mockito)	83	117	117	+18 casos
Componente / contrato HTTP (WebMvcTest)	12	12	12	—
Integração e sistema (Spring Boot + PostgreSQL)	137	166	166	+19 casos (CT-CONC-001 agora repete 10 vezes)
Aceitação BDD (Cucumber / Gherkin)	16	18	18	—
Total	**248**	**313**	**313 (100%)**	**+37 casos, +48 execuções**

Suítes novas: `RobustezSegurancaIT` (10), `ConfiguracoesAlternativasIT` (2), `InicializadorGerenteTest` (6).

11.3 Cobertura de código (JaCoCo)

Contador	Ciclo 1	Ciclo 1.1	Meta
Instruções	93,6%	**96,6%** (8.294 / 8.587)	—
Linhas	94,1%	**96,8%** (1.499 / 1.548)	≥ 80% ✅
Ramos	81,2%	**87,6%** (408 / 466)	≥ 70% ✅
Métodos	92,0%	**95,0%** (509 / 536)	—

Por pacote (linhas / ramos): `venda` 99,3% / 94,9% · `caixa` 98,1% / 100% · `cliente` 98,7% / 89,7% · `auth` 98,6% / 95,8% · `comum` 97,8% / 93,5% · `cupom` 97,8% / 84,2% · `usuario` 97,7% / 91,2% · `auditoria` 100% / 87,5% · `produto` 94,8% / 77,6% · `seguranca` 93,4% / 73,9% · `comum.erro` 86,7% / 75,0%. **Todos os pacotes estão acima da meta de 70% de ramos.**

11.4 Teste de mutação (PIT)

Escopo ampliado para 12 classes (entrou `InicializadorGerente`).

Rodada	Mortos	Sobreviventes	Sem cobertura unitária	Força dos testes
4 (após as mudanças do ciclo)	196	13	86	93,8%
5	214	2	79	99,1%
6 (final)	**216**	**0**	79	**100%** ✅

As 13 lacunas estão em Defeitos, seção 5.1 (MUT-08 a MUT-13). Escore sobre todos os mutantes gerados: 216 ÷ 295 = 73,2% (era 60,2%). Fonte: evidencias/mutacao-pit.csv.

11.5 Desempenho e carga (VAL-07 — limite de 3.000 ms)

Caso	Operação	Amostras	Média	Mediana	p95	Máximo	Resultado
CT-PERF-001	`GET /produtos` (1.000 produtos, limite 100)	20	41,0 ms	35 ms	74 ms	77 ms	✅
CT-PERF-002	`GET /produtos?nome=…`	20	25,1 ms	22 ms	36 ms	46 ms	✅

Caso	Operação	Amostras	Média	Mediana	p95	Máximo	Resultado
CT-PERF-003	`POST /vendas` com 10 itens	30	45,5 ms	47 ms	59 ms	62 ms	✅
CT-PERF-004	`GET /produtos/{id}`	30	7,2 ms	7 ms	9 ms	9 ms	✅
CT-PERF-005a	Carga: primeira rajada, 20 operadores simultâneos (consulta + venda)	40	108,4 ms	77 ms	248 ms	284 ms	✅
CT-PERF-005b	Carga: regime, 20 simultâneos, 400 requisições, **148 req/s**, 0 erros	400	100,9 ms	61 ms	281 ms	981 ms	✅

No CT-PERF-005 o critério é mais rígido que nos demais: **toda** requisição (não só o p95) abaixo de 3 s e nenhuma resposta de erro. Antes da correção da DEF-WEB-006 a pior requisição da primeira rajada chegava a 4,0 s.

11.6 Concorrência

CT-CONC-001 (20 vendas simultâneas do último lote de 5) passou a repetir **10 vezes** por execução: 10/10 com exatamente 5 vendas, 15 rejeições e estoque final 0. CT-SEG-019 mostrou que 10 senhas erradas enviadas ao mesmo tempo para o mesmo CPF não escapam do limite (contagem atômica no banco).

11.7 Execução em servidor real

A API subiu como servidor HTTP (modo demonstração, porta 8080) e passou num roteiro de 14 passos com `curl`: health check, Swagger (23 rotas), login dos dois perfis, venda do exemplo do contrato (R$ 215,00, comissão R$ 10,75), bloqueio por estoque (422), sangria negada ao vendedor (403), fechamento sem diferença, bloqueio de login (429, `Retry-After: 899`), logout (token passa a 401) e cabeçalhos de segurança. Evidência: evidencias/smoke-servidor-real.txt.

11.8 Critérios de saída — reavaliação

#	Critério	Meta	Ciclo 1.1	Situação
CS-1	Casos P1 executados e aprovados	100%	100%	✅
CS-2	Taxa de aprovação	≥ 95%	100% (313/313)	✅
CS-3	Defeitos Críticos/Altos abertos (backend)	0	0	✅
CS-4	Requisitos do escopo com ≥ 1 caso aprovado	100%	100%	✅
CS-5	Cobertura	≥ 80% linhas, ≥ 70% ramos	96,8% / 87,6%	✅
CS-6	Tempo de resposta	p95 < 3.000 ms	p95 máx. 281 ms; pior requisição 981 ms	✅ (ambiente local)
CS-7	Força dos testes unitários	≥ 80%	100%	✅

11.9 Conclusão do Ciclo 1.1

Os quatro riscos residuais de software do Ciclo 1 foram corrigidos e cada correção tem caso automatizado que falharia sem ela. **O que ainda separa o backend da produção é infraestrutura, não código:** subir com Docker Compose (PEND-03), medir a carga no ambiente alvo (PEND-04) e instalar o certificado TLS no proxy (PEND-07). As sessões exploratórias (PEND-01) e o reteste do desktop (PEND-02) seguem abertos e não bloqueiam.

12. Ciclo 1.2 — Banco de dados

Objetivo: fazer do banco a última barreira — o histórico financeiro e de auditoria não pode ser adulterado nem por erro da aplicação nem por SQL direto — e preparar o ambiente Docker de produção. **Execução oficial:** 28/10/2026, 11:49:26 – 11:53:05 · `./mvnw clean verify` · **320/320** · BUILD SUCCESS.

12.1 O que mudou

Item	Implementação	Casos
Imutabilidade	Migração V4: gatilhos tornam auditoria, itens de venda e movimentações de caixa "só inclusão"; venda não é apagada e só muda de CONCLUIDA para CANCELADA; caixa fechado não muda	CT-DB-007, 008, 010
Consistência	CHECKs de cancelamento completo, fechamento de caixa, subtotal do item, comissão, UF e documentos só com dígitos	CT-DB-009, 010
Desempenho	8 índices em chaves estrangeiras e em `(status, data_venda)`	regressão de desempenho
Menor privilégio	Papéis `pdv_migracao` (Flyway) e `pdv_app` (API, sem DDL, com limites de tempo); `PUBLIC` sem acesso	CT-DB-011 a 013
Docker	Checksums de dados, SCRAM-SHA-256, log de consultas lentas, porta 5432 só local, health check da API, rotação de logs, serviço de **backup diário verificado** com retenção de 7 dias, scripts de backup/restauração e `.env` com senhas geradas	Roteiro RT-DEP-01 a 07

`PapeisDoBancoIT` executa o mesmo `docker/postgres/papeis.sql` do container no PostgreSQL embarcado e sobe a API completa com `pdv_app` e o Flyway com `pdv_migracao` — a configuração do Docker fica testada mesmo sem Docker.

12.2 Resultados

Indicador	Ciclo 1.1	Ciclo 1.2
Casos distintos	248	**255** (+7: CT-DB-007 a 013)
Execuções	313	**320** (0 falhas)
Cobertura de linhas / ramos	96,8% / 87,6%	96,6% / 87,6%
Carga (CT-PERF-005b)	p95 281 ms, máx. 981 ms	p95 190 ms, máx. 1.565 ms, 149 req/s, 0 erros

Ajustes de teste: `AuditoriaIT` (CT-AUD-005) desliga o gatilho só durante o preparo do cenário que envelhece um log; `BancoDeDadosIT` (CT-DB-003) passou a fechar o caixa com data, como exige o novo CHECK.

12.3 Execução no Docker (roteiro RT-DEP)

Docker 29.9 / Compose 5.6 com WSL 2, 28/10/2026, 14:19 – 14:21 · evidência completa.

Caso	Resultado	Status
RT-DEP-01	`banco` e `api` healthy, `backup` no ar; V1–V4 aplicadas	✅
RT-DEP-02	`/actuator/health` → `{"status":"UP"}`	✅
RT-DEP-03	Login do gerente antes e depois de `restart api`; 1 gerente no banco	✅
RT-DEP-04	Logout 200; token antigo após reinício → 401	✅
RT-DEP-05	Proxy Caddy local + `PDV_EXIGIR_HTTPS=true`: HTTP → 302 HTTPS; HTTPS 200 com HSTS, TLS 1.3 (evidência)	✅ (local; repetir na AWS)
RT-DEP-06	Backup manual verificado com `pg_restore --list`; backup automático "backup ok"	✅

Caso	Resultado	Status
RT-DEP-07	Venda feita depois do backup some na restauração (vendas 2 → 1, estoque 8 → 10); API UP	✅

A primeira execução encontrou dois defeitos de script, corrigidos e retestados na mesma data: **DEF-WEB-007** (restauração falhava no Git Bash e deixava a API parada) e **DEF-WEB-008** (`subir-docker.sh` pedia a senha do banco e terminava com erro). Ao preparar o proxy TLS, uma inspeção encontrou o **DEF-WEB-009** (o health check falharia com HTTPS obrigatório). Detalhes no Relatório de Defeitos.